



DIAGNOSTICO ARQUITECTONICO

Estudiante: Francisco R. Diaz

Matrícula: 2023-3300016

Asignatura: Ingeniería de Software

Profesor/a: Ing. Carlos Artemio Escalante Lorenzo

Fecha: 3/07/2026

1. Estilo Arquitectonico Actual

El sistema implementa **Arquitectura Hexagonal (Ports & Adapters)** combinada con principios de DDD (Domain-Driven Design) en el backend, y una SPA con React + Vite en el frontend. No es codigo espagueti ni MVC basico.

1.1 Estructura de Capas del Backend

```
backend/src/
  domain/          <- Nucleo: entidades, value objects, estados
    orden/         <- Aggregate Root: OrdenServicio (patron State)
    presupuesto/   <- CalculadoraOrden (patron Strategy)
    servicio/      <- ServicioFactory (patron Factory Method)
  application/     <- Casos de uso + TallerFacade + EventBus
    TallerFacade.js <- Patron Facade: punto unico de entrada
    EventBus.js    <- Patron Observer: eventos de dominio
    casos-de-uso/  <- 13 casos de uso independientes
  infrastructure/  <- Repositorios Prisma/PostgreSQL
  interfaces/http/ <- Controllers Express (solo delegan al Facade)
    controllers/   <- Sin logica de negocio
    routes/        <- Definicion de rutas REST
    middleware/    <- Auth JWT, manejo de errores
```

1.2 Patrones de Diseno Implementados

Patron	Clase	Proposito
State	OrdenServicio + estados/	Ciclo de vida de la orden (8 estados)
Facade	TallerFacade.js	Punto unico de acceso para controladores
Builder	OrdenServicioBuilder.js	Construccion controlada de ordenes
Factory Method	ServicioFactory.js	Creacion de servicios por tipo
Strategy	CalculadoraOrden.js	Estrategias de cálculo y descuento
Observer	EventBus.js	Notificaciones por eventos de dominio
Repository	infraestructura/	Acceso a datos desacoplado del dominio

2. Tres Lugares con Logica de Negocio Mezclada

Problema 2.1 - Validacion de negocio en NuevaOrden.jsx

Archivo: frontend/src/pages/NuevaOrden.jsx - Lineas 70-93

El frontend realiza validaciones de campos obligatorios (marca, modelo, placa, anio) y conversiones de tipo (parseInt) que son reglas de negocio. Estas deberian vivir en el backend dentro de un CrearVehiculoUseCase. Si mañana cambia la validacion, hay que editar el frontend.

```
// PROBLEMA: Validacion de negocio en componente de presentacion
if (!nuevoVehiculo.marca || !nuevoVehiculo.modelo ||
    !nuevoVehiculo.placa || !nuevoVehiculo.anio) {
  setErrorVehiculo("Marca, modelo, anio y placa son obligatorios.")
  return // <- Esta regla deberia estar en el backend
}
anio: parseInt(nuevoVehiculo.anio), // <- Conversion de tipo: rol del
backend
```

Problema 2.2 - Formato de moneda y calculos repetidos en vistas

Archivo: frontend/src/pages/Ordenes.jsx - Linea 94

La funcion formatCurrency y el calculo del subtotal (reduce) se repiten en Ordenes.jsx y Facturacion.jsx. Si cambia la moneda o el locale hay que editar multiples archivos del frontend. Esta logica deberia centralizarse en un utility o retornarse ya calculada desde el backend.

```
// PROBLEMA: Logica financiera repetida en la vista
const formatCurrency = (val) =>
  new Intl.NumberFormat("es-DO", { style: "currency", currency: "DOP" })
    .format(Number(val) || 0) // Duplicada en Ordenes.jsx y Facturacion.jsx

// Calculo de subtotal directamente en JSX:
{formatCurrency(detalleData.servicios.reduce((a,s) => a + Number(s.costo),
0))}
```

Problema 2.3 - Logica de autorizacion por roles en Layout.jsx

Archivo: frontend/src/components/Layout.jsx - Lineas 16-19

La logica de control de acceso (que menus ve cada rol) esta directamente dentro del componente de navegacion visual. Si se agrega un nuevo rol (ej: SUPERVISOR), hay que editar el JSX del Layout en lugar de un archivo de configuracion separado.

```
// PROBLEMA: Autorizacion mezclada con presentacion
...(user?.rol === "ADMIN" ? [ // <- Logica de roles en el componente visual
  { name: "Usuarios", path: "/usuarios", ... },
  { name: "Equipo Tecnico", path: "/mecanicos", ... }
] : []),
```

3. Tres Malas Prácticas en el Código

Mala Practica 3.1 - alert() y confirm() nativos en produccion

Archivo: frontend/src/pages/Ordenes.jsx - Lineas 51, 63, 67

Los alert() y confirm() bloquean el event loop del navegador, no se pueden personalizar visualmente, rompen la experiencia de usuario y en algunos navegadores modernos estan bloqueados en iframes. Ademas, mostrar error.message directamente expone detalles internos del sistema al usuario final.

```
// MAL: alert() y confirm() en produccion
if (!window.confirm("Generar la factura final?")) return
alert("Orden eliminada exitosamente")
alert("Error al avanzar el estado: " + error.message) // expone detalles
internos

// SOLUCION: Implementar componente ConfirmModal y sistema Toast
notifications
```

Mala Practica 3.2 - useEffect sin cleanup y dependencias incompletas

Archivo: frontend/src/pages/Dashboard.jsx - Lineas 11-14

El array de dependencias vacio [] "engana" a React. Si fetchOrdenes cambia (ej: cuando cambia el token de autenticacion), el efecto no se re-ejecuta. Sin una funcion return de cleanup, puede haber memory leaks en aplicaciones de larga duracion cuando el componente se desmonta con peticiones pendientes.

```
// MAL: useEffect sin cleanup y sin dependencias reales
useEffect(() => {
  fetchOrdenes() // No esta en el array de dependencias
  fetchEstadisticas() // No esta en el array de dependencias
}, []) // Array vacio: solo se ejecuta una vez, puede quedar stale

// Sin return () => { controller.abort() } para cancelar peticiones
```

Mala Practica 3.3 - Endpoint stub agregarServicio en produccion

Archivo: backend/src/interfaces/http/controllers/OrdenController.js - Lineas 61-68

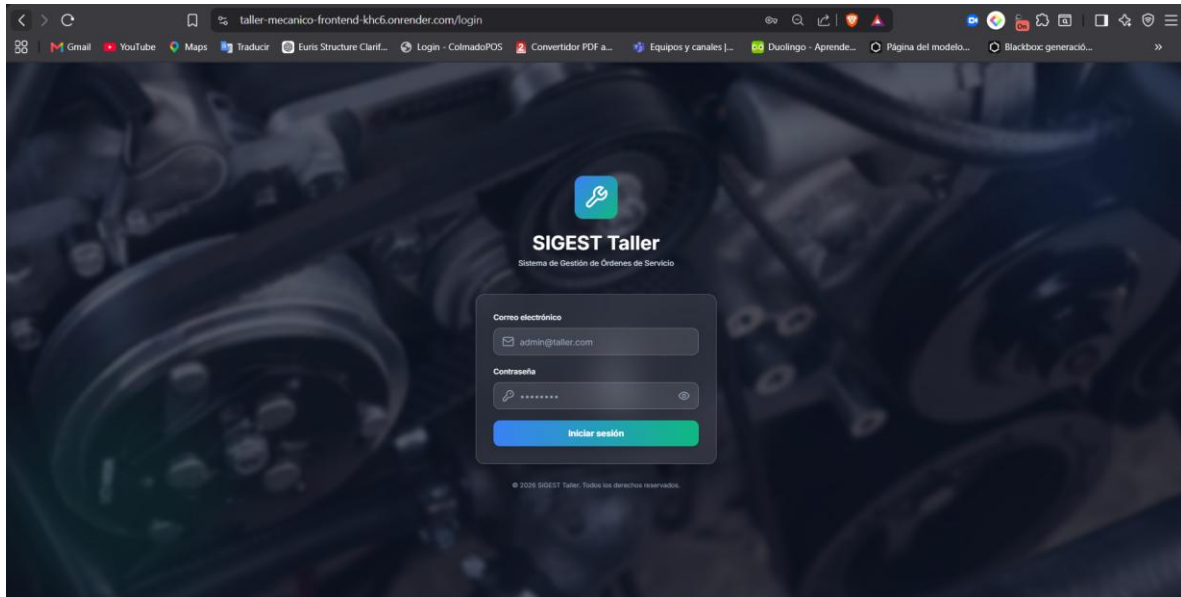
Un endpoint que siempre devuelve { success: true, message: "Servicio agregado" } sin realizar ninguna accion real es un stub que llego a produccion. Retorna exito sin hacer nada, lo que puede causar inconsistencias silenciosas en los datos si algun cliente confia en que este endpoint funciona.

```
// MAL: Metodo stub - dice exito pero no hace nada
async agregarServicio(req, res, next) {
  const orden = await tallerFacade.obtenerOrden(req.params.id);
  // Agregar servicio... delegar a use case <- TODO en produccion
  res.json({ success: true, data: null, message: "Servicio agregado" });
  // Siempre dice exito aunque no hizo nada
}
```

```
// SOLUCION: Implementar AgregarServicioUseCase o retornar 501 Not Implemented
```

4. Tres Problemas de Flujo Visual y Organizacion

Pantalla de Login (referencia)



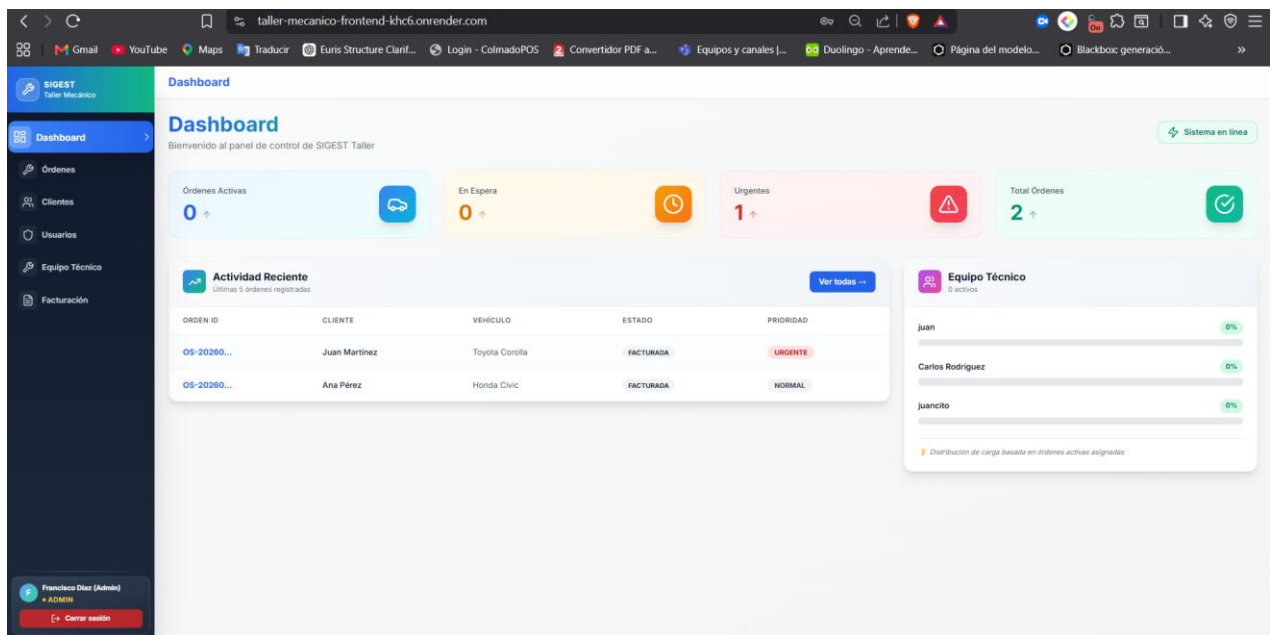
Problema Visual 1 - Titulo H1 duplicado en el Dashboard

PROBLEMA: El título "Dashboard" aparece DOS veces en la misma pantalla:

1. En el header superior de navegacion (texto pequeno, gradiente)
2. Como H1 del contenido principal (texto grande, 36px, gradiente)

Esto viola la jerarquia visual unica (una página debe tener un solo H1 significativo) y desperdicia espacio vertical valioso. El usuario lee "Dashboard" dos veces al escanear la pantalla, creando ruido visual innecesario.

Codigo afectado: Layout.jsx linea 134 + Dashboard.jsx linea 75



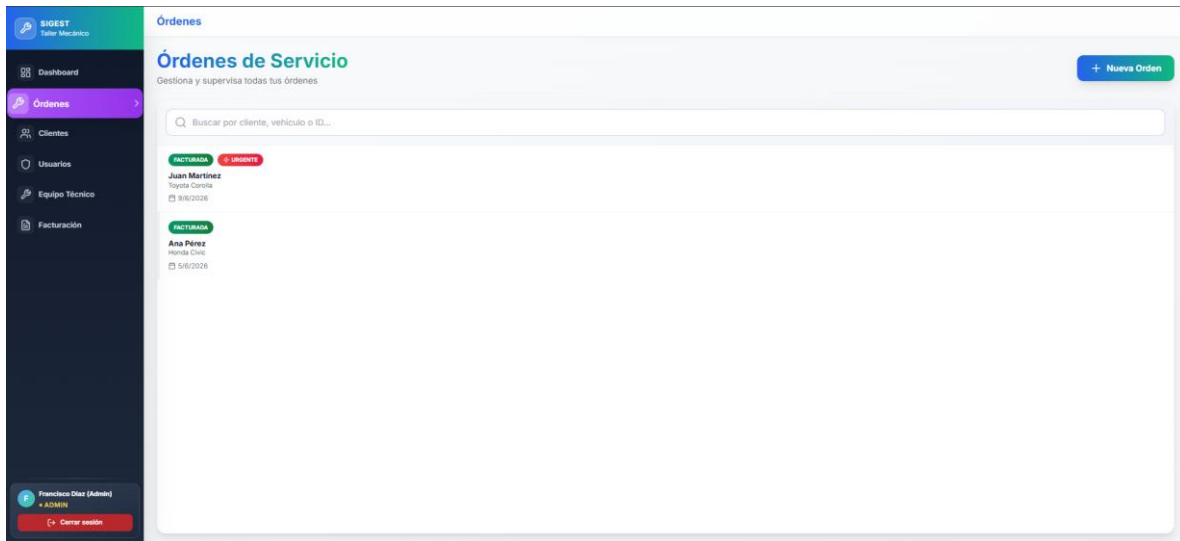
Problema Visual 2 - Identificadores de ordenes ilegibles

PROBLEMA: En la tabla "Actividad Reciente" del Dashboard y en la lista de Ordenes, el identificador de cada orden se muestra como un código truncado (ej: "OS-20260..."). Este ID no tiene significado para un recepcionista o mecánico que necesita referenciar una orden

verbalmente por teléfono, en papel o en comunicación con el cliente.

Un numero de orden legible como #0001 o ORD-2026-042 sería mucho más útil.

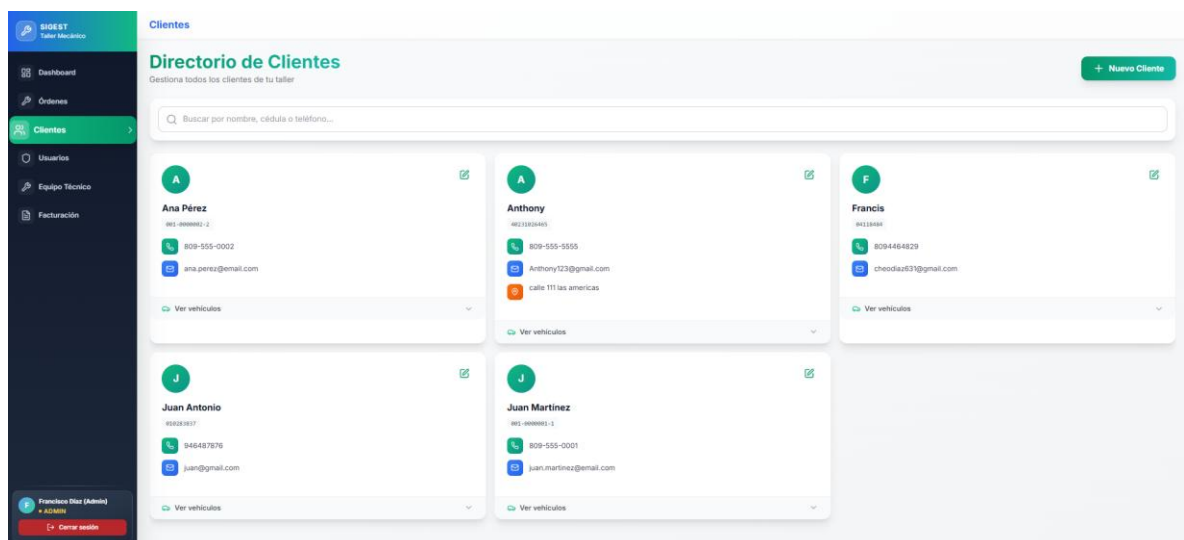
Código afectado: Dashboard.jsx linea 170 + Ordenes.jsx linea 198



Problema Visual 3 - Inconsistencia de patrones visuales entre modulos

PROBLEMA: La pantalla de Clientes usa tarjetas (cards en grid 3 columnas) mientras que Ordenes usa una lista vertical de items. Esta inconsistencia en el patron visual entre modulos rompe la coherencia de la interfaz.

Adicionalmente, en la página de Ordenes, cuando el usuario selecciona una orden, la lista de la izquierda se comprime al 40%% del ancho (w-2/5), dejando los nombres de clientes y vehiculos truncados e ilegibles en resoluciones de 1280px o menores (comunes en computadoras de taller).



El usuario no puede anticipar como se presentara la informacion en cada seccion, lo que aumenta la curva de aprendizaje del sistema.

Codigo afectado: Ordenes.jsx linea 120 (w-2/5 dinamico)

2 — Plan de mejora (individual)

1. Reorganización del código en capas

El proyecto será refactorizado adoptando la Arquitectura Hexagonal y Principios DDD (Domain-Driven Design). Las capas se organizarán de la siguiente manera:

- **Capa de Dominio (Modelos):** Se definirá el Aggregate Root "OrdenServicio", sus entidades y Value Objects correspondientes. Toda lógica pura que muta el estado vivirá aquí.
- **Capa de Aplicación (Casos de Uso):** Se crearán clases específicas para orquestrar la lógica de negocio (ej. CrearVehiculoUseCase), aislándolas completamente de la infraestructura o la web.
- **Capa de Infraestructura:** Manejará el acceso a base de datos mediante repositorios y la inyección de dependencias a través de patrones como Facade.
- **Capa de Interfaces (Controladores):** Los controladores Express únicamente se encargarán de recibir la petición HTTP y delegar su procesamiento al TallerFacade.

2. Lógica de negocio a mover de los controladores a clases de servicio

- **Lógica de validación:** Las restricciones (ej. obligatoriedad de placa, marca, modelo y transformaciones de datos) dejarán de existir en las vistas Frontend o Controladores y se moverán al Caso de Uso o Servicio pertinente.
- **Generación de Órdenes y Facturas:** Se implementará el patrón State en la capa de Dominio para manejar la lógica compleja de transiciones entre estados y el patrón Builder para su creación. El controlador delegará directamente estas complejas estructuras al servicio en lugar de resolverlas él mismo.

3. Mejoras al frontend (Flujo visual claro y consistente)

- **Organización de Menús:** Se abstraerá la configuración de navegación dinámica a un archivo dedicado que renderizará los elementos de menú basándose en el rol del usuario (ADMIN, MECANICO, CAJERO).
- **Experiencia de Usuario (UX) y Notificaciones:** Se sustituirán las primitivas bloqueantes del navegador (alert y confirm) por componentes customizados asíncronos (Toast y ConfirmModal), ofreciendo al usuario retroalimentación sin interrumpir su navegación.
- **Jerarquía Visual:** Se eliminarán los encabezados H1 redundantes que saturan la vista de los componentes secundarios.
- **Centralización Lógica:** Funciones de formateo (moneda y fechas) se centralizarán para

garantizar que la presentación del dinero sea consistente en todos los módulos (Órdenes, Facturación, Dashboard).

4. Prácticas de calidad a incorporar

- **Control de Versiones con Git:** Se utilizará Git realizando commits atómicos, aislados y con mensajes descriptivos. Los cambios de frontend y backend se implementarán en ramas y envíos independientes.
- **Validaciones en Backend:** En lugar de confiar únicamente en el Frontend, el Backend implementará validaciones rigurosas tanto a nivel de esquemas de API como a través de reglas de dominio enriquecidas.
- **Arquitectura Robusta:** Utilización de herramientas de control de dependencias para certificar que el Dominio y la Aplicación nunca llamen a herramientas de Infraestructura, asegurando código limpiamente testeable.